

# **Tiny TAC**

Syntax, Semantics, Demo

# Tiny TAC

**Tiny TAC** (`ttac`) is the small language used in the VCGen notes.

It keeps only the semantic core needed for examples:

- typed registers: `bool`, `int`, `bytemap`
- assignments, `havoc`, `assume`, `assert`
- basic blocks with explicit terminators
- phi nodes at joins

The goal is not to document `ctac` syntax. The goal is to make the VCGen algorithms precise.

# Types And Expressions

Registers have one of three types:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap}$$

The SMT model is intentionally direct:

$$\text{int} \mapsto \text{Int}$$

$$\text{bytemap} \mapsto \text{Int} \rightarrow \text{Int}$$

Integer expressions:

$$\begin{aligned} i ::= & n \mid x \mid M[i] \\ & \mid i + i \mid i - i \mid i * i \mid i / i \\ & \mid \text{ite}(b, i, i) \end{aligned}$$

Boolean expressions:

$$\begin{aligned} b ::= & \text{true} \mid \text{false} \mid c \\ & \mid i < i \mid i \leq i \mid i = i \mid b = b \\ & \mid \neg b \mid b \wedge b \mid b \vee b \\ & \mid \text{ite}(c, b, b) \end{aligned}$$

`ite` is an expression operator, not a structured statement.

# Maps

Loads and stores are expressions over bytemaps.

```
M := havoc  
i := havoc  
v := havoc  
x := M[i]  
M2 := M[i := v]
```

Semantically,  $M_2$  agrees with  $M$  at every address except  $i$ , where it stores  $v$ .

# Commands

Command forms:

```
x := e  
x := havoc  
x := phi [B1: x1, B2: x2]  
assume b  
assert c
```

`:=` is assignment. Equality is a separate expression/operator.

Example:

```
x := havoc  
y := havoc  
z := havoc  
c := x < y  
assume y < z  
assert c
```

`assume` may use an expression. `assert` uses a named bool register.

# Blocks And Control

A complete program — `safe_core.ttac`, the running example of this deck:

```
entry:
  M := havoc
  i := havoc
  limit := havoc
  x := M[i]
  y := x + 1
  M2 := M[i := y]
  c := y <= limit
  if c goto ok else bad

ok:
  assert c
  goto exit
```

```
bad:
  assume not c
  goto exit

exit:
  halt
```

Unstructured basic blocks; branches are terminators and branch on a **named** bool register.

Both branches rejoin at `exit` — every block is on an entry-to-exit path. The `assert` in `ok` only runs on the `c`-true path.

# Phi Nodes

Phi nodes select by predecessor block.

```
L:
  x1 := 1
  goto J

R:
  xr := 2
  goto J

J:
  x := phi [L: x1, R: xr]
  goto exit
```

- Enter from L:  $x := x_l$
- Enter from R:  $x := x_r$

## Demo: First Look At A Program

ttac stats reads the surface — commands, types, memory capability:

```
$ ttac stats safe_core.ttac -plain
overview.blocks: 4      overview.commands: 9
command_kinds.Assign: 4  Havoc: 3  Assert: 1  Assume: 1
types.int: 4            types.bool: 1  types.bytemap: 2
memory.capability: bytemap-rw  loads: 1  updates: 1
shape.asserts: 1
```

Four blocks, one assert, read-write bytemap use — matches the program on the previous slide.

# Demo: Semantics, Executed

The interpreter **is** the operational semantics. Zero-havoc run:

```
$ ttac run safe_core.ttac -trace
status: done (finished)    assert_ok: 0    assert_fail: 0
entry:
  M := havoc                # havoc bytemap
  i := havoc = 0
  limit := havoc = 0
  x := M[i] = 0             # M[0]
  y := x + 1 = 1
  M2 := M[i := y]           # M[0 := 1]
  c := y <= limit = false
  if c goto ok else bad    # c=false -> bad
bad:
  assume not c              # assume: true
  goto exit                # -> exit
exit:
  halt                     # halt
```

## Demo: Semantics, Executed

The interpreter **is** the operational semantics. Zero-havoc run:

```
$ ttac run safe_core.ttac -trace
status: done (finished)    assert_ok: 0    assert_fail: 0
entry:
  M := havoc                # havoc bytemap
  i := havoc = 0
  limit := havoc = 0
  x := M[i] = 0              # M[0]
  y := x + 1 = 1
  M2 := M[i := y]            # M[0 := 1]
  c := y <= limit = false
  if c goto ok else bad      # c=false -> bad
bad:
  assume not c               # assume: true
  goto exit                  # -> exit
exit:
  halt                       # halt
```

Havoc defaulted to 0, so `c` is false and execution takes the `bad` path — the assert never runs. One execution, not all of them.

## Demo: Asking About All Executions

assert c in block ok — can it **ever** fail?

```
$ ttac vcgen safe_core.ttac -solve  
unsat
```

**UNSAT** no execution reaches ok with c false — the branch guard protects the assert on **every** path.

# Demo: Asking About All Executions

assert  $c$  in block  $ok$  — can it **ever** fail?

```
$ ttac vcgen safe_core.ttac -solve  
unsat
```

**UNSAT** no execution reaches  $ok$  with  $c$  false — the branch guard protects the assert on **every** path.

That is the VCGen contract, seen end to end:

$VCGen(P)$  is satisfiable  $\Leftrightarrow P$  has an assertion-failure execution

Interpreter: one chosen execution. Solver: a question over all of them.